

Aether — Zero-Downtime Migration

Cross-Runtime Migration Strategies

16 runtime pairs, health gates, 30s drain cap, automatic rollback — see
Migration Internals for honest limits on volumes and DNS.

Universal Runtime Control Plane

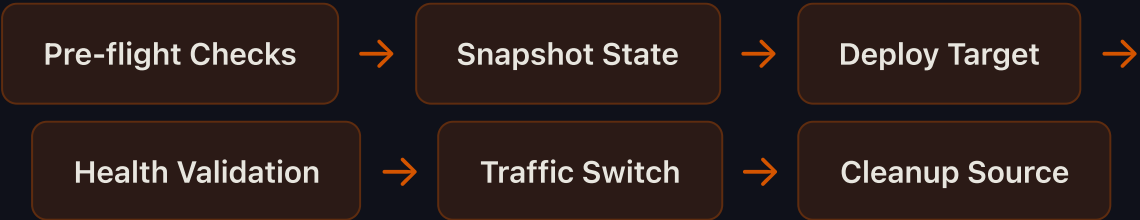
Migration Overview

Aether supports migrations across all 4 runtimes in every direction — 16 total runtime pair combinations managed through a unified interface.

Runtime Migration Matrix

From \ To	Podman	Kubernetes	KubeVirt	Metal3
Podman	Same-runtime guard Supported	Supported	Supported	Supported
Kubernetes	Supported	Same-runtime guard Supported	Supported	Supported
KubeVirt	Supported	Supported	Same-runtime guard Supported	Supported
Metal3	Supported	Supported	Supported	Same-runtime guard Supported

Migration Lifecycle



Key Principles

- Every migration begins with a state snapshot for guaranteed rollback capability
- Health validation uses exponential backoff to avoid overwhelming targets

- Same-runtime migrations are blocked — use `aether deploy` instead
- All migration events are recorded in the tamper-evident audit trail

16

Runtime Pair
Combinations

3

Migration
Strategies

0

Downtime (Blue-
Green)

30s

Max Backoff Cap

Immediate Strategy

The fastest migration path — stop the source, start the target, validate, and update state. Ideal for development and staging environments.

Migration Timeline

1

Stop Source Workload

Gracefully stop the workload on the current runtime. Connections are drained and the container or VM is halted.

2

Start Target Workload

Deploy the workload to the destination runtime using the same YAML spec with runtime-specific adaptations.

3

Validate Health

Run health checks with exponential backoff. If validation fails, automatic rollback restores the source.

4

Update State

Persist the new runtime assignment to Aether's state file. The migration is now complete.

Use Cases



Development Environments

Fast iteration when testing workloads across different runtimes. Brief downtime is acceptable.



Staging Migrations

Validate runtime compatibility before production. Quick feedback loop on migration feasibility.



Maintenance Windows

Scheduled migrations during planned downtime. Simplest strategy with minimal resource overhead.



Stateless Services

Services that can tolerate brief interruption without data loss or session impact.

Command Example

```
# Migrate a workload from its current runtime to Kubernetes using the immediate strategy
$ aether migrate my-app kubernetes --strategy immediate
```

```
Migrating 'my-app' to kubernetes (strategy: immediate)...
```

```
[1/4] Stopping source workload on podman...
```

```
[2/4] Deploying to kubernetes...
```

```
[3/4] Validating health (attempt 1/5)...
```

```
[4/4] Updating state...
```

```
Migration complete: my-app now running on kubernetes
```

Blue-Green Strategy

Zero-downtime migration with full validation before traffic switch. The target workload runs alongside the source until health is confirmed.

Migration Timeline

1

Start Target Workload (Green)

Deploy the workload to the destination runtime while the source (Blue) continues serving traffic.

2

Validate Target Health

Run comprehensive health checks against the Green deployment. Exponential backoff ensures thorough validation.

3

Switch Traffic

Once Green passes all health checks, update routing to point to the new runtime. Zero-downtime cutover.

4

Stop Source (Blue)

Gracefully decommission the Blue deployment. State is updated to reflect the new runtime assignment.

Advantages



Zero Downtime

Traffic is only switched after the target is fully healthy. Users experience no interruption during migration.



Full Validation

The target workload is validated in a live environment before receiving production traffic.



Instant Rollback

If the Green deployment fails health checks, the Blue deployment is still running. No rollback delay.



Comparison Testing

Both deployments run simultaneously, allowing side-by-side performance and behavior comparison.

Command Example

```
# Blue-green migration to KubeVirt with zero downtime
$ aether migrate my-app kubevirt --strategy blue-green

Migrating 'my-app' to kubevirt (strategy: blue-green)...
[1/4] Deploying green instance on kubevirt...
[2/4] Validating green health (attempt 1/5)...
[3/4] Switching traffic to green...
[4/4] Stopping blue instance on kubernetes...
Migration complete: my-app now running on kubevirt (zero downtime)
```

Rolling Strategy

Gradual replica migration with health checks at every step. Traffic shifts incrementally from source to target, minimizing blast radius.

Migration Timeline

1

Deploy Initial Canary (10%)

Start a small subset of replicas on the target runtime. Route 10% of traffic to validate basic functionality.

2

Health Check + Scale (25%)

If the canary passes health validation, increase the target allocation to 25%. Monitor error rates and latency.

3

Progressive Expansion (50% / 75%)

Continue scaling the target while reducing the source. Each step includes full health validation with exponential backoff.

4

Complete Migration (100%)

All traffic is now on the target runtime. Source replicas are decommissioned and state is updated.

Exponential Backoff Health Validation

Each health check attempt uses exponential backoff to avoid overwhelming the target runtime during startup:

Delay Formula: $\text{delay} = \text{base} * 2^{\text{attempt}}$ (capped at 30 seconds)

Attempt 1: $1s * 2^0 = 1s$

Attempt 2: $1s * 2^1 = 2s$

Attempt 3: $1s * 2^2 = 4s$

Attempt 4: $1s * 2^3 = 8s$

Attempt 5: $1s * 2^4 = 16s$ (max 5 attempts by default)

Canary Configuration

```
# Rolling migration with custom canary steps
$ aether migrate my-app metal3 --strategy rolling \
  --canary-steps 10,25,50,75,100 \
  --health-interval 5s \
  --health-retries 5
```

Migrating 'my-app' to metal3 (strategy: rolling)...

Step 1/5: 10% on metal3 – health OK

Step 2/5: 25% on metal3 – health OK

Step 3/5: 50% on metal3 – health OK

Step 4/5: 75% on metal3 – health OK

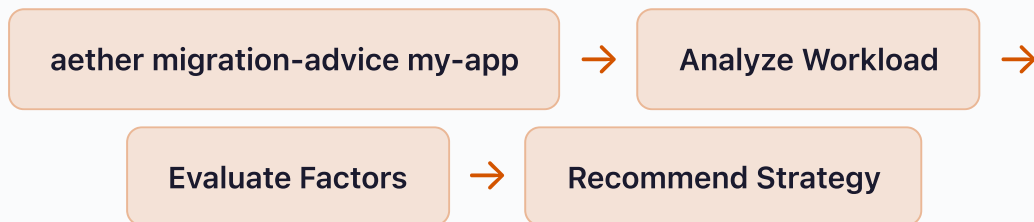
Step 5/5: 100% on metal3 – health OK

Rolling migration complete: my-app fully on metal3

Migration Advice AI

Aether analyzes workload characteristics to recommend the optimal migration strategy — no guesswork required.

How It Works



Analysis Factors



Resource Usage

CPU, memory, and storage requirements determine whether the target runtime can handle the workload. High-resource workloads favor rolling migrations.



Network Dependencies

Services with complex networking (ingress, service mesh, DNS) benefit from blue-green to validate connectivity before switch.



Persistence

Workloads with persistent volumes or stateful data require careful migration. Blue-green ensures data integrity before cutover.



GPU Requirements

GPU-bound workloads have limited scheduling options. Advice factors in GPU availability across target runtimes.

Example Output

```
$ aether migration-advice my-app
```

Migration Advice for 'my-app'

=====

```
Current runtime:  podman
Replicas:        3
Persistent vols:  1 (data-vol, 50Gi)
Network deps:     ingress, service-mesh
GPU:              none
```

Recommended strategy: blue-green

Reasoning:

- Persistent volume requires data validation before cutover
- Network dependencies need live connectivity testing
- Multi-replica workload benefits from parallel validation
- No GPU constraints limiting target scheduling

Alternative: rolling (if gradual traffic shift is preferred)

Avoid: immediate (persistent data risk without validation)

Safety Features

Every migration is protected by multiple safety layers — from automatic rollback to state snapshots and health validation.



Automatic Rollback

If health validation fails at any stage, Aether automatically restores the source workload from the pre-migration snapshot. No manual intervention required.



Health Validation with Exponential Backoff

Health checks use the formula **base * 2^attempt** with a 30-second cap. This prevents overwhelming newly-started workloads while ensuring thorough validation.



State Snapshots Before Migration

A complete state snapshot is taken before every migration. This includes workload configuration, runtime assignment, and deployment metadata for guaranteed recovery.



Same-Runtime Guard

Aether blocks migrations where source and target runtimes are identical. This prevents accidental re-deployments and ensures migrations are intentional runtime changes.

Rollback in Action

```
$ aether migrate my-app kubevirt --strategy blue-green
```

```
Migrating 'my-app' to kubevirt (strategy: blue-green)...
```

```
[1/4] Deploying green instance on kubevirt...
```

```
[2/4] Validating green health (attempt 1/5)...  
Health check failed (attempt 1/5), retrying in 2s...  
Health check failed (attempt 2/5), retrying in 4s...  
Health check failed (attempt 3/5), retrying in 8s...  
Health check failed (attempt 4/5), retrying in 16s...  
Health check failed (attempt 5/5) – max retries exceeded  
Rolling back: restoring source workload from snapshot...  
Rollback complete: my-app restored on kubernetes
```

Safety Guarantees

Feature	Protection	Trigger
Automatic Rollback	Restores source from snapshot	Health validation failure
Exponential Backoff	Prevents target overload	Every health check cycle
State Snapshots	Guaranteed recovery point	Before migration starts
Same-Runtime Guard	Blocks no-op migrations	Pre-flight validation
Audit Logging	Complete migration history	Every migration event

Get Started with Migrations

Three commands to master cross-runtime migration.

Start Migrating Today

Move workloads between Podman, Kubernetes, KubeVirt, and Metal3 with confidence.

Essential Commands

```
# Get migration advice for a workload
$ aether migration-advice my-app

# Migrate with a specific strategy
$ aether migrate my-app kubernetes --strategy blue-green

# Migrate with immediate strategy (dev/staging)
$ aether migrate my-app podman --strategy immediate

# Rolling migration with custom canary steps
$ aether migrate my-app metal3 --strategy rolling \
  --canary-steps 10,25,50,75,100

# View migration history
$ aether audit --filter migration
```

[Documentation](#)

[Community](#)

Full migration guide with runtime-specific considerations and troubleshooting.

Share migration patterns and learn from other Aether operators in the community.